ESTUDO EM DIA

Documento completo de especificação do produto e engenharia

Versão: 1.0 — 02 de novembro de 2026
Produto: Estudo em Dia | Academia Viva (decorativa)
Status: especificação para implementação; não é aplicação já desenvolvida.

> **Síntese:** plataforma universal para sessões de foco com recompensas, registro manual de conteúdos, revisão periódica, preservação do ponto de retorno e crescimento automático de uma academia 2D. Arquitetura React/TypeScript + NestJS + PostgreSQL/Prisma. Nenhum banco obrigatório de exercícios e nenhuma LLM rodando no app.

ESTUDO EM DIA — DOCUMENTAÇÃO MESTRA

Versão: 1.0 (especificação proposta)
Data-base: 02/11/2026
Status: pronto para implementação incremental; parâmetros propostos ainda precisam de validação com usuários.
Produto: Estudo em Dia, com gamificação visual “Academia Viva”.

1. Como usar este pacote

Este pacote é a referência para implementação pelo Codex. Leia antes de gerar código:

1. 01_PRODUTO_ESCOPO.md — visão, público, jornadas, limites e decisões.
2. 02_REQUISITOS_REGRAS.md — requisitos identificados, regras e aceitação.
3. 03_ARQUITETURA_DADOS.md — stack, componentes, modelo relacional e segurança.
4. 04_API_ALGORITMOS.md — endpoints, contratos e regras determinísticas.
5. 05_UX_GAMIFICACAO.md — telas, fluxos, foco, continuidade, academia.
6. 06_TESTES_QUALIDADE.md — estratégia, cenários e definição de pronto.
7. 07_PLANO_CODEX.md — ordem de execução, prompts, orçamento de escopo.
8. 08_SUBMISSAO_WORKSHOP.md — regras do trabalho, evidências e entrega.
9. 09_MATRIZ_RASTREABILIDADE.md — ligação entre regras, fluxos e testes.
10. AGENTS.md — instruções resumidas para o agente de programação.

O arquivo DOCUMENTACAO_COMPLETA.md concatena estes materiais para leitura linear. O DOCX e o PDF são versões para apresentação e consulta; **os arquivos Markdown são a referência operacional para o Codex**.

2. Resumo executivo

Problema: as pessoas têm dificuldade de iniciar e sustentar sessões de estudo, lembrar o que estudaram e retomar uma tarefa após interrupções. Muitas ferramentas gerenciam horários ou oferecem conteúdo próprio, mas não integram foco, memória do contexto e revisão de assuntos escolhidos pelo estudante.

Solução: aplicativo web responsivo com cadastro de qualquer disciplina e conteúdo, sessões de foco recompensadas, registro manual do que foi estudado, algoritmo de revisões periódicas, pontos de retorno e uma academia decorativa que evolui com a atividade registrada.

Promessa realista: facilitar a continuidade e a organização da prática, não medir diretamente a atenção cerebral nem certificar domínio sobre temas arbitrários.

Arquitetura: React + TypeScript + Vite + Tailwind CSS; API REST NestJS; PostgreSQL; Prisma ORM; autenticação por sessão segura; testes automatizados; Docker Compose para o banco no desenvolvimento.

3. Decisões de produto — confirmadas no brainstorming

Identificador	Decisão	Origem
DEC-01	O usuário pode estudar **qualquer assunto**; não haverá catálogo obrigatório nem exercícios prontos.	Confirmada
DEC-02	O usuário é recompensado por **sessões de foco registradas**.	Confirmada
DEC-03	O usuário pode registrar manualmente conteúdos estudados.	Confirmada
DEC-04	Os conteúdos registrados geram revisões periódicas com autoavaliação.	Confirmada
DEC-05	O app preserva um ponto de retorno para continuar de onde parou.	Confirmada
DEC-06	A Academia Viva é **decorativa**, não um simulador de cidade com economia.	Confirmada
DEC-07	A solução terá backend próprio, API e banco relacional; não será somente armazenamento local.	Confirmada
DEC-08	Não haverá dependência de LLM, tokens de API ou plataformas de exercícios.	Confirmada

4. Parâmetros de projeto — propostas técnicas, ajustáveis

Valores de XP, intervalos, limites de tempo, estágios da academia, política de autenticação, nomes de telas e duração das sessões constituem **propostas de implementação**, não decisões do usuário já validadas. Servem para eliminar ambiguidades e permitir testes repetíveis. Alterações devem ser registradas em ADR/CHANGELOG, refletidas nos testes e, quando necessário, acompanhadas de migração de banco.

5. Não objetivos do MVP

- Não criar exercícios ou trilhas de conteúdo próprios.
- Não corrigir respostas abertas automaticamente, avaliar código alheio ou usar LLM.
- Não declarar que a pessoa **de fato** ficou atenta apenas porque um timer esteve ativo.
- Não medir “nível de TDAH”, capacidade cognitiva, inteligência ou domínio científico de um assunto.
- Não construir recursos 3D, cidade editável, economia, chat, ranking social ou marketplace.
- Não entregar sincronização offline completa nem app mobile nativo no MVP.

6. Definição de produto minimamente completo

Uma pessoa consegue criar conta, registrar uma disciplina, iniciar e concluir uma sessão de foco, receber XP uma única vez, registrar o conteúdo aprendido, obter uma revisão para data futura, realizar a revisão com autoavaliação, retomar uma tarefa interrompida, visualizar o crescimento decorativo da academia e encontrar todos os dados preservados após entrar novamente. A aplicação deve ser executável com instruções reproduzíveis e testes centrais aprovados.

01 — VISÃO DO PRODUTO E ESCOPO

1.1 Visão

Estudo em Dia é uma aplicação web para estudantes autônomos de qualquer área. Ela ajuda a começar, sustentar, registrar e retomar atividades de estudo e organiza revisões de conteúdos cadastrados pelo próprio usuário. O progresso aparece também por meio da **Academia Viva**, uma cena decorativa com estágios desbloqueados.

Proposta de valor: "Estude o que quiser. Continue de onde parou. Revise na hora certa. Veja seu esforço virar progresso."

1.2 Problema e oportunidade

Uma mesma pessoa pode estudar em livros, cursos, vídeos, aulas presenciais, ambientes virtuais, código e cadernos. O aplicativo não deve exigir que ela transfira todo o material para dentro dele. Deve guardar o contexto mínimo necessário: *o que estou estudando; o que fiz; onde parei; qual é meu próximo passo; o que preciso revisar*.

O produto integra cinco ciclos: **planejar de maneira leve → executar foco → registrar conteúdo → revisar periodicamente → retomar sem recomeçar do zero**. A academia é consequência visual do uso, não uma tarefa administrativa adicional.

1.3 Público e cenários

- Universitário que estuda disciplinas diferentes e precisa acompanhar o avanço.
- Autodidata que alterna cursos, vídeos, projetos, livros e resolução externa de exercícios.
- Pessoa que interrompe estudos frequentemente e precisa preservar um ponto de retorno.
- Concurseiro que registra tópicos de edital e revisões sem importar um banco de questões.
- Estudante de idiomas que registra regras, vocabulário, aulas e material externo.

O aplicativo não substitui professor, terapeuta, especialista em aprendizagem nem ferramenta de correção acadêmica.

1.4 Personas ilustrativas, não dados reais

Persona A — autodidata: faz curso de programação em plataforma externa; quer preservar a página, o repositório e a próxima ação.
Persona B — universitária: estuda por livros e apostilas; quer cadastrar capítulos e revisar conceitos.
Persona C — rotina irregular: tem dias curtos; quer iniciar uma atividade pequena sem acumular dívida de tarefas.

1.5 Objetivos de produto e indicadores

Objetivo	Indicador	Interpretação responsável
Facilitar o começo	Tempo entre abrir a página e iniciar uma atividade	Mede interação, não foco mental
Facilitar continuidade	Taxa de retomada depois de ausência e uso de ponto de retorno	Métrica comportamental
Favorecer revisões	Revisões realizadas / revisões vencidas em janela móvel	Não prova retenção real sem avaliação externa
Gerar hábito útil	Dias com sessões concluídas em 28 dias, sem punição por falhas	Evitar pressão por streak perfeito
Gerar feedback visível	Estágios da academia liberados com eventos auditáveis	Não é diploma nem certificado

1.6 Escopo fechado do MVP (P0)

1. Cadastro, login, logout, perfil básico e fuso horário do usuário.
2. CRUD de disciplinas independentes de qualquer catálogo.
3. CRUD de conteúdos estudados: título, disciplina, notas simples, material/URL opcional, dificuldade autodeclarada e status.
4. Sessão de foco: iniciar, pausar, retomar, concluir, interromper; recuperação de sessão pendente; registro de tempo efetivo e XP.
5. Ponto de retorno: contexto, posição e próxima ação; criação/edição e exibição no dashboard; persistência entre acessos.
6. Revisões periódicas por conteúdo, com data, estado, registro histórico, avaliação subjetiva e próximo agendamento.
7. Dashboard com ação principal, revisão vencida/próxima, resumo de foco e academia.
8. Academia decorativa 2D com quatro estágios, calculada por eventos de XP e marcos de revisão.
9. Estatísticas básicas por período e por disciplina.
10. Segurança mínima, migrações, testes centrais, documentação de execução e dados de demonstração fictícios opcionais.

1.7 Escopo desejável (P1, somente após P0 estável)

- Missões por tamanho: essencial, completa e profunda; sugestões sem bloquear liberdade de escolha.
- Botão “Não consigo começar” com quatro respostas simples.
- Pesquisa e filtros avançados de conteúdos.
- Preferências para intervalo de revisões e objetivos semanais.
- Exportação em JSON e CSV do próprio usuário.
- Decorações extras e marcos visuais de continuidade.
- Notificações **dentro do app** de revisões pendentes.

1.8 Fora do escopo inicial (P2)

- Aplicativo Android/iOS nativo, modo offline com sincronização e conflitos.
- Integrações com calendário, plataformas de cursos, Gmail, IA generativa ou serviços pagos.
- Correção automática de respostas discursivas ou aplicação de provas reais.
- FSRS calibrado com grandes históricos, predição individual de memória ou algoritmos diagnósticos clínicos.
- Comunidade, amigos, social, placar competitivo, assinaturas e pagamento.

- Cidade 3D, drag-and-drop de prédios, moeda virtual e gerenciamento de recursos.

1.9 Jornada ponta a ponta

1. **Primeiro acesso:** usuário se cadastra e define fuso horário; pode criar disciplina “História”.
2. **Preparação:** cadastra atividade “Ler capítulo 3” ou inicia foco ligado à disciplina sem atividade pré-cadastrada.
3. **Foco:** define 25 minutos; inicia; pode pausar ou interromper; recebe indicação de tempo real registrado.
4. **Encerramento:** recebe XP pelo tempo elegível, vê a evolução visual; decide se quer registrar conteúdo ou continuar depois.
5. **Registro:** salva “Revolução Francesa: causas sociais”, notas e material; escolhe próxima ação “Revisar páginas 48–55”.
6. **Retorno:** ao entrar novamente, dashboard mostra a próxima ação e permite abrir material externo.
7. **Revisão:** no vencimento, usuário tenta recordar sem consultar notas; revela-as quando desejar; avalia “não lembrei”, “difícil”, “bem” ou “fácil”; novo vencimento é calculado.
8. **Progresso:** histórico, XP e academia refletem apenas eventos efetivamente registrados.

1.10 Princípios de UX

- **Sem cadastro exaustivo:** mínimo de campos obrigatórios.
- **Sem obrigação de planejar tudo:** usuário pode começar um foco sem ter criado atividade/conteúdo.
- **Sem recompensa enganosa:** marcar minutos não equivale a comprovar concentração.
- **Sem punição por ausência:** não criar dívida cumulativa nem resetar conquistas por faltar.
- **Sem interrupção durante foco:** recompensas só aparecem em transições.
- **Controle e transparência:** usuário vê o porquê de cada revisão e pode corrigir conteúdo.

1.11 Critérios de sucesso da apresentação

O demonstrador executa, em sequência, uma conta de teste, criação de disciplina, sessão curta com tempo de teste controlado, registro de conteúdo, exibição da revisão agendada, realização de uma revisão forçada em dados fictícios (sem mexer no relógio real), ponto de retorno e mudança de estágio. Deve ser possível demonstrar idempotência: repetir conclusão não concede XP extra.

1.12 Dependências e premissas

- Internet e backend disponíveis para usar a versão hospedada.
- PostgreSQL acessível pelo backend.
- Os dados pertencem à conta autenticada.
- Fuso padrão America/Sao_Paulo, alterável pelo usuário; armazenar instantes em UTC.
- Avisos e cronogramas de revisão são orientações, não garantia de memorização.

02 — REQUISITOS FUNCIONAIS, REGRAS DE NEGÓCIO E ACEITAÇÃO

2.1 Convenções

- RF-xxx: requisito funcional; RN-xxx: regra de negócio; RNF-xxx: requisito não funcional; CA-xxx: critério de aceitação.
- P0: obrigatório no MVP; P1: melhoria desejável; P2: futuro.

- Regras P0 são normativas para esta versão **proposta**, ainda sujeitas à revisão do responsável pelo produto.
- Todos os registros pertencem a `userId`; toda operação protegida exige autenticação e validação de propriedade **no servidor**.
- "Conteúdo" significa um tema registrado pelo usuário, não um exercício fornecido pela plataforma.

2.2 Requisitos — conta e perfil

ID	Prioridade	Descrição verificável
RF-001	P0	Cadastrar conta com nome, e-mail normalizado e senha segura.
RF-002	P0	Autenticar credenciais e iniciar sessão com cookie protegido.
RF-003	P0	Encerrar sessão e invalidá-la no servidor.
RF-004	P0	Consultar o próprio perfil e alterar nome e fuso horário válido.
RF-005	P0	Impedir acesso aos dados de outra conta mesmo com ID conhecido.
RF-006	P1	Permitir exportar os próprios dados em formato portátil.
RF-007	P2	Recuperação de senha por e-mail, verificação de e-mail e MFA.

2.3 Requisitos — disciplinas e conteúdos

ID	Prioridade	Descrição verificável
RF-010	P0	Criar, listar, renomear, arquivar e consultar disciplinas próprias.
RF-011	P0	Criar conteúdo com título, disciplina, notas opcionais, link e data de estudo.
RF-012	P0	Editar, consultar, pesquisar por texto simples e arquivar conteúdo.
RF-013	P0	Iniciar revisão periódica de conteúdo quando explicitamente solicitado no cadastro ou depois.
RF-014	P0	Permitir registrar conteúdo após foco, mas **não tornar o cadastro obrigatório** para obter XP.
RF-015	P0	Distinguir conteúdos ativos, arquivados e sem revisão habilitada.
RF-016	P1	Classificar conteúdos com etiquetas e filtros por data.
RF-017	P2	Anexos, importação de fontes externas e edição em lote.

2.4 Requisitos — foco

ID	Prioridade	Descrição verificável
RF-020	P0	Escolher duração prevista entre 5 e 120 minutos; padrões de 10, 25 e 50.
RF-021	P0	Iniciar sessão vinculada

ID	Prioridade	Descrição verificável
		opcionalmente a disciplina e a conteúdo/atividade.
RF-022	P0	Pausar e retomar com persistência da máquina de estados.
RF-023	P0	Concluir uma sessão ou encerrá-la como interrompida, registrando tempo efetivo.
RF-024	P0	Conceder XP de foco apenas por evento válido e sem duplicidade.
RF-025	P0	Exibir cronômetro correto após refresh e calcular tempo a partir de timestamps do servidor.
RF-026	P0	Detectar sessão inconclusa na volta e solicitar confirmação de conclusão parcial/abandono.
RF-027	P0	Não exibir gamificação intrusiva na tela de foco.
RF-028	P1	Registrar interrupções e motivo opcional ao concluir.

2.5 Requisitos — continuidade

ID	Prioridade	Descrição verificável
RF-030	P0	Registrar ponto de retorno: onde parei e próxima ação objetiva.
RF-031	P0	Associar ponto de retorno opcionalmente a disciplina, conteúdo e URL do material.
RF-032	P0	Exibir ponto de retorno aberto e ação de retomada na página inicial.
RF-033	P0	Editar, concluir ou substituir ponto de retorno; preservar data de atualização.
RF-034	P0	Recuperar contexto mesmo após logout, refresh ou novo login no mesmo/novo dispositivo.
RF-035	P1	Oferecer um único toque “Não consigo começar” e resposta de baixa fricção.
RF-036	P1	Sugerir um ponto de retorno ao fechar uma sessão, sem exigir preenchimento.

2.6 Requisitos — revisões periódicas

ID	Prioridade	Descrição verificável
RF-040	P0	Listar conteúdos com revisão vencida, de hoje e futura.
RF-041	P0	Agendar a primeira revisão automaticamente para conteúdo com revisão habilitada.
RF-042	P0	Oferecer passo “Tente lembrar” antes de mostrar a anotação.

ID	Prioridade	Descrição verificável
RF-043	P0	Permitir revelar as próprias notas e classificar recordação em quatro graus.
RF-044	P0	Calcular próxima data de revisão segundo regra determinística e auditável.
RF-045	P0	Armazenar histórico de cada revisão realizada, sem sobrescrever registros anteriores.
RF-046	P0	Impedir duas revisões processadas para o mesmo vencimento pela mesma requisição repetida.
RF-047	P0	Permitir suspender e reativar revisão de conteúdo; suspensão não remove histórico.
RF-048	P1	Personalizar intervalos e horário preferido de revisão.
RF-049	P2	Algoritmos mais sofisticados calibrados por desempenho, sem promessas de diagnóstico.

2.7 Requisitos — gamificação e estatísticas

ID	Prioridade	Descrição verificável
RF-050	P0	Consultar XP total, XP de foco e XP de revisões com origem auditável.
RF-051	P0	Conceder XP de revisão somente pela primeira resposta válida do ciclo.
RF-052	P0	Calcular estágio da academia automaticamente por XP acumulado.
RF-053	P0	Mostrar 4 estágios decorativos estáticos/SVG sem edição de mapa.
RF-054	P0	Manter conquistas e XP mesmo após faltar dias de estudo.
RF-055	P0	Exibir minutos registrados, sessões finalizadas e revisões por período.
RF-056	P1	Liberar estilos visuais e pequenas decorações por marcos definidos.
RF-057	P2	Social, competição, cidade editável, moedas, construções manuais.

2.8 Regras de negócio detalhadas

Contas e propriedade

RN-001 E-mail é único após normalização por trim e lowercase. A resposta de erro de login não revela se o usuário existe.
RN-002 Senhas são armazenadas apenas por hash de algoritmo apropriado (Argon2id preferencial), nunca texto puro.
RN-003 Recursos de usuário A não podem ser consultados/alterados por usuário B: preferir consulta com where: {id, userId}.
RN-004 Alterar o fuso não altera instantes históricos UTC; pode alterar o dia local de agregação de

relatórios futuros, com critério documentado.
RN-005 Em P0, arquivamento lógico preserva histórico; exclusão física fica fora do MVP.

Disciplinas, conteúdos e pontos de retorno

RN-010 Disciplina exige nome de 1–80 caracteres após trim; sem duplicata de mesmo nome ativo por usuário (comparação normalizada).
RN-011 Conteúdo exige título de 1–160 caracteres; notas limitadas a 10.000 caracteres; URL opcional http(s), validada, nunca executada nem inserida como HTML.
RN-012 Um conteúdo pode não ter revisão habilitada. Habilitar revisão cria exatamente um agendamento ativo, e desabilitar cancela esse agendamento sem apagar histórico.
RN-013 Conteúdo arquivado não aparece na fila ativa; seu histórico fica consultável.
RN-014 Um usuário pode manter mais de um ponto de retorno, mas o dashboard destaca apenas o mais recentemente atualizado que esteja OPEN.
RN-015 Ponto de retorno exige "próxima ação" com 1–500 caracteres; "onde parei" pode ter até 2.000; link opcional. Pode existir sem conteúdo vinculado.
RN-016 Não usar IA para deduzir notas, aprender o conteúdo ou corrigir afirmações inseridas pelo usuário.

Sessões de foco e cálculo de tempo

RN-020 Duração planejada: inteiro 5–120 minutos; no máximo uma sessão RUNNING/PAUSED/NEEDS_RECONCILIATION por usuário.
RN-021 Estados: RUNNING, PAUSED, COMPLETED, INTERRUPTED, DISCARDED, NEEDS_RECONCILIATION; transições autorizadas no capítulo 4.
RN-022 O servidor registra timestamps das mudanças de estado; o cliente não envia tempo "ganho" nem XP para persistência.
RN-023 Tempo acumulado de foco = soma dos segmentos ativos entre START/PAUSE/RESUME/END, limitado ao planejado; valores negativos são rejeitados e horários de cliente não têm autoridade.
RN-024 O cronômetro pode continuar enquanto o usuário estuda em plataforma externa; aba invisível **não** implica pausa automática. Não afirmar detecção física de atenção.
RN-025 Uma sessão aberta além da duração planejada + 15 minutos não é automaticamente premiada. É colocada em NEEDS_RECONCILIATION no próximo acesso/ação, com escolha de confirmar período estudado (até o limite plausível registrado) ou descartar.
RN-026 Para sessão interrompida, é permitido registrar tempo parcial já acumulado; não há promessa de minutos exatos de concentração.
RN-027 XP de foco = floor(segundosElegíveis/60), com mínimo de 5 minutos efetivos para gerar XP, limite de **60 XP por sessão** e **180 XP por dia local do usuário**. O restante do tempo é preservado nas estatísticas sem conceder XP adicional.
RN-028 Um evento FOCUS_SESSION_REWARD é único por sessão, protegido por transação e índice único. Repetições de POST, timeout e múltiplas abas não geram XP duplo.
RN-029 Finalização salva sessão, segmentos, XP e evento de progresso atomicamente. Falha parcial deve fazer rollback.
RN-030 Expiração da sessão de login não apaga dados; o usuário autentica novamente antes de confirmar alterações.

Revisões

RN-040 Uma revisão ativa por conteúdo; conteúdo sem revisão habilitada não entra na fila.
RN-041 Primeira revisão vence no **dia local seguinte** ao cadastro/habilitação; mostrar a data ao usuário.

RN-042 Escala de intervalos proposta em dias: [1, 3, 7, 14, 30, 60]; armazenar stepIndex (0 a 5). O valor é heurístico e configurável em versão futura, não é garantia de retenção.
RN-043 Autoavaliação: AGAIN (não lembrei), HARD (com dificuldade), GOOD (bem), EASY (fácil). É avaliação subjetiva; a interface não declara correção objetiva.
RN-044 Transição de intervalos: AGAIN → índice 0; HARD → índice atual; GOOD → min(indice+1,5); EASY → min(indice+2,5). Agendar dataLocalDaRevisão + intervalos[novoIndice] dias civis; ver detalhes em 04.
RN-045 Revisões atrasadas não criam múltiplas cobranças: exibir apenas **um** cartão vencido por conteúdo. Não penalizar XP, estágio nem histórico por atraso.
RN-046 A data futura é calculada a partir do dia da realização, não da data original vencida; não gerar revisão no passado por atraso.
RN-047 Revisão e atualização de nextDueDate ocorrem em transação; usar reviewAttemptId/chave idempotente para não recompensar duas vezes.
RN-048 XP de revisão: 5 XP por conclusão de revisão válida, limitado a **50 XP por dia local**, sem recompensa por apenas abrir o cartão. Repetir requisição não rende novo XP.
RN-049 "Revelar notas" pode ser opcional; permitir classificar mesmo sem notas preenchidas, pois o assunto pode existir em fonte externa.
RN-050 Arquivar conteúdo ou desabilitar revisão não destrói o histórico.

Academia, recompensa e métricas

RN-060 XP é calculado exclusivamente no backend a partir de eventos persistidos e auditáveis; cliente apenas apresenta.
RN-061 XP_total = soma(XP de eventos válidos). Não usar moedas nem gasto de XP.
RN-062 Estágios decorativos propostos: **I** 0–99 XP; **II** 100–349 XP; **III** 350–799 XP; **IV** a partir de 800 XP. O palco muda automaticamente, sem seleção de edifícios.
RN-063 Mudança de estágio nunca remove XP; nenhuma ausência gera perda de construção.
RN-064 Relatórios distinguem *tempo registrado* de *aprendizado verificado*. Em P0, o último não é calculado.
RN-065 Um dia local é identificado pelo fuso IANA salvo na conta. O limite diário de XP usa a data local do evento, mesmo quando o instante UTC cai em outro dia.

2.9 Requisitos não funcionais (RNF)

ID	P0/P1	Exigência / critério de verificação
RNF-001	P0	API REST com prefixo /api/v1, JSON, códigos HTTP coerentes.
RNF-002	P0	Frontend responsivo (largura de 360px a desktop), navegação por teclado e labels acessíveis.
RNF-003	P0	TypeScript strict, DTOs validados, sem any indiscriminado.
RNF-004	P0	PostgreSQL com migrações versionadas, chaves estrangeiras e índices para consultas comuns.
RNF-005	P0	Auth segura, rate limit para login, headers de segurança, cookies HttpOnly e proteção CSRF/Origin.
RNF-006	P0	Nenhum segredo ou credencial real no repositório. .env.example apenas com placeholders.

ID	P0/P1	Exigência / critério de verificação
RNF-007	P0	Idempotência de encerramento e revisão; transações para operações com XP.
RNF-008	P0	Tempo persistido sobreviver a recarregamento do navegador; estado reconstruído do servidor.
RNF-009	P0	Endpoint de saúde, logging estruturado sem passwords e mensagens de erro consistentes.
RNF-010	P0	Testes automatizados sobre regras centrais; comando para rodá-los localmente.
RNF-011	P0	Dados separados por usuário; teste de autorização cruzada obrigatório.
RNF-012	P0	Endpoint e UI não apresentam desempenho/retorno como diagnóstico clínico ou prova de conhecimento.
RNF-013	P1	Metas iniciais de experiência: interações usuais responsivas em ambiente local; medir antes de afirmar SLA.
RNF-014	P1	Exportação de dados do usuário; política de privacidade e exclusão definitiva antes de lançamento público real.
RNF-015	P1	CI rodando lint, typecheck, testes e build.

2.10 Critérios de aceitação (formato dado / quando / então)

CA-001 — Primeiro acesso: dado visitante sem conta, quando cadastra dados válidos, então cria conta e é levado ao dashboard autenticado; e-mail duplicado retorna conflito controlado.
CA-002 — Isolamento: dado usuário B, quando solicita disciplina de A por ID, então recebe 404 ou 403 sem divulgar conteúdo privado.
CA-003 — Disciplina universal: dado usuário, quando cria “Direito Constitucional”, então pode registrar conteúdos sem escolher catálogo ou curso predefinido.
CA-004 — Foco avulso: dado usuário sem disciplina, quando inicia e encerra foco válido, então sessão é preservada e XP é concedido sem exigir conteúdo cadastrado.
CA-005 — Pausa: dado foco em andamento, quando pausa por 60s e retoma, então os 60s não contam como tempo ativo.
CA-006 — Atualização: dado foco ativo, quando recarrega a página, então timer e estado refletem registros do servidor, sem zerar ou duplicar a sessão.
CA-007 — Encerramento duplicado: dado sessão concluída, quando requisição é reenviada, então total de XP e contador de conclusões não aumentam.
CA-008 — Sessão pendente: dado navegador fechado antes de concluir, quando retorna, então vê fluxo de reconciliação, não XP inventado.
CA-009 — Conteúdo: dado foco concluído, quando omite registro de conteúdo, então ainda pode voltar ao dashboard.
CA-010 — Agenda inicial: dado conteúdo cadastrado com revisão ativa hoje, quando salva, então aparece uma revisão para amanhã na data local.

CA-011 — Revisão com dificuldade: dada revisão cujo índice é 3, quando responde AGAIN, então novo índice é 0 e vencimento é dia seguinte.
CA-012 — Revisão bem-sucedida: dado índice 1, quando responde GOOD, então índice vai a 2 e agenda +7 dias civis.
CA-013 — Revisão repetida: dada avaliação processada, quando repete mesma chave idempotente, então recebe mesmo resultado e nenhum novo XP.
CA-014 — Atraso: dado conteúdo vencido há 10 dias, quando abre fila, então há um único item e nenhum bloqueio por backlog.
CA-015 — Continuidade: dado ponto de retorno aberto, quando reloga, então vê "onde parei" e "próxima ação" salvos.
CA-016 — Academia: dado total XP de 350, quando abre academia, então visualiza estágio III; depois de uma ausência continua no mesmo estágio.
CA-017 — Limite diário: dado XP diário já no limite de foco, quando completa nova sessão, então tempo ainda conta na estatística, mas XP diário não ultrapassa 180.
CA-018 — Dados de teste: dados exemplos fictícios, quando são carregados por seed de demonstração, então a aplicação opera sem credenciais reais nem dependência externa.

2.11 Tratamento de exceções e invariantes

- Backend deve rejeitar transições inválidas (retomar sessão concluída; concluir sessão de outra pessoa; editar item inexistente) com erros estruturados.
- Em falha de rede, frontend não declara sucesso até resposta confirmada; pode mostrar estado de envio e permitir retry idempotente.
- Se o relógio do dispositivo for alterado, ele não muda XP calculado pelo servidor; o timer usa o timestamp retornado pelo backend para reconciliar exibição.
- Não aceitar duração negativa, superior a máximo, texto vazio obrigatório, datas inválidas ou `userId` enviado pelo cliente como autoridade.
- Conteúdo de notas é tratado como texto; escape de HTML para evitar XSS; links externos com comportamento seguro.
- Alterações de fuso usam a regra do dia civil local e são testadas perto da meia-noite e transições DST.
- Em conflitos de duas abas, backend é a fonte de verdade; retornar 409 e estado atual quando necessário.

03 — ARQUITETURA TÉCNICA, DADOS E SEGURANÇA

3.1 Decisão de arquitetura (ADR-001)

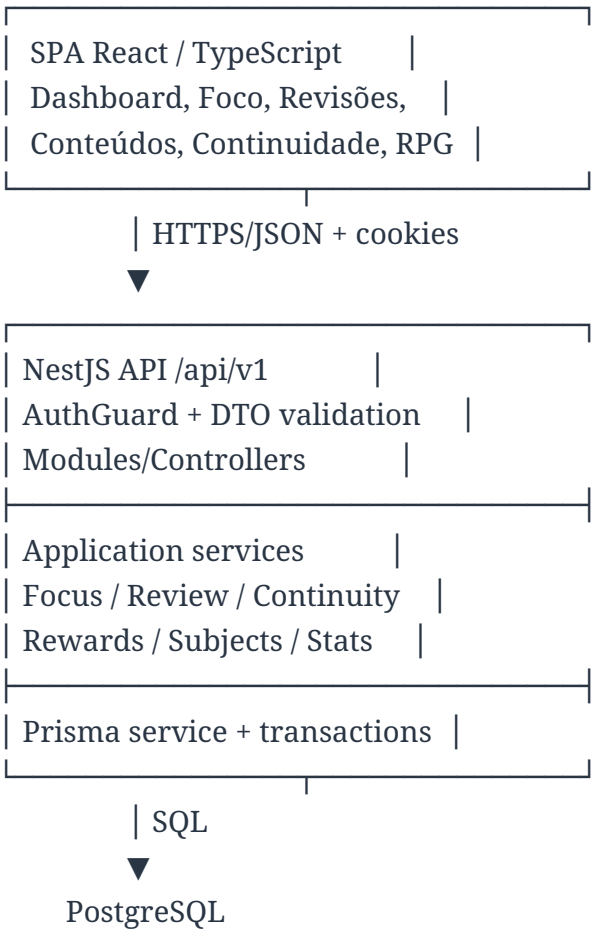
Padrão: monólito modular com frontend separado, API REST e banco PostgreSQL. Evitar microsserviços e filas distribuídas no MVP.

Camada	Escolha	Decisão prática
UI	React + TypeScript + Vite	SPA responsiva, telas modulares, hooks pequenos
CSS	Tailwind CSS + CSS/SVG próprios	Academia em SVG estático/camadas, sem engine de jogo
Navegação	React Router	Rotas públicas e privadas
API	Node.js + NestJS	Controllers, serviços, DTOs, Guards, OpenAPI
Persistência	PostgreSQL	Relacional, transações, constraints e

Camada	Escolha	Decisão prática
		índices
ORM	Prisma 7 + @prisma/adapter-pg	Fixar versões compatíveis; Prisma 7 exige adapter e ESM
Autenticação	JWT curto em cookie HttpOnly + sessão revogável	Não salvar token em localStorage; consultar sessão ativa
Testes	Vitest frontend e Jest/Supertest backend	Cobrir regras, endpoints e autorização
Desenvolvimento	npm workspaces + Docker Compose (Postgres)	Monorepo sem turborepo/Nx inicialmente
Documentação de API	OpenAPI/Swagger via NestJS	Contratos gerados e exemplos verificáveis

Justificativa: uma única linguagem em todas as camadas reduz o custo de contexto do Codex, sem remover a separação de responsabilidades. Prisma deve ser configurado conforme documentação da versão fixada (Prisma 7 usa driver adapter e configuração moderna); não misturar snippets de versões 5/6/7/8.

3.2 Desenho dos componentes



3.3 Estrutura proposta do repositório

```
estudo-em-dia/
  AGENTS.md
  README.md
  .env.example
```

```
package.json                # npm workspaces
docker-compose.yml          # PostgreSQL local
docs/                       # este pacote de especificação
apps/
  web/
    src/
      app/                  # rotas, providers
      components/           # base UI
      features/
        auth/ subjects/ contents/ focus/ reviews/
        continuity/ academy/ statistics/
      lib/api.ts
      styles/
    public/academy/         # 4 assets SVG/PNG próprios
  api/
    prisma/
      schema.prisma
      migrations/
      seed.ts
    src/
      auth/ users/ subjects/ contents/
      focus/ reviews/ continuity/ rewards/ statistics/
      common/               # config, guards, errors, time
      prisma/
    test/
packages/
  contracts/                # tipos DTO compartilhados (opcional P1)
```

Em P0, **não** criar `packages/contracts` se isso atrasar o trabalho. É aceitável consumir OpenAPI ou manter DTOs pequenos e testes de contrato sem duplicação desnecessária.

3.4 Responsabilidades e dependências

- `auth`: cadastro, login, logout, cookie, proteção por sessão.
- `users`: perfil, fuso IANA.
- `subjects`: disciplina e arquivamento.
- `contents`: conteúdo livre e regras de edição/arquivamento.
- `focus`: estados de sessão, intervalos ativos, reconciliação.
- `reviews`: agendamento, tentativas, cálculo da próxima revisão.
- `continuity`: pontos de retorno.
- `rewards`: fonte única de XP, limites e consulta de eventos.
- `statistics`: agregações filtradas por usuário e fuso.
- `academy`: **somente leitura derivada** do total de XP; não precisa de tabela de posições de edifícios.

As regras de negócio ficam nos serviços do backend, não no frontend. A interface é consumidora de dados e fonte de ações do usuário, não autoridade para saldo de XP ou cálculo final de tempo.

3.5 Modelo relacional lógico

Tipos: IDs UUID gerados no servidor; datas de eventos como timestamptz/DateTime UTC; vencimento de revisão como data civil (DATE) sem horário; durações inteiras em segundos; updatedAt e createdAt nas entidades mutáveis.

User

Campo	Tipo	Restrição
id	UUID	PK
name	varchar(100)	obrigatório
emailNormalized	varchar(254)	único, lowercase
passwordHash	text	obrigatório, nunca resposta API
timeZone	varchar(80)	IANA, padrão America/Sao_Paulo
createdAt / updatedAt	datetime UTC	obrigatório

AuthSession

Campo	Tipo	Restrição
id	UUID	PK, também jti de JWT
userId	UUID	FK User
expiresAt	datetime UTC	obrigatório
revokedAt	datetime UTC	nulo quando ativa
createdAt	datetime UTC	obrigatório

Subject

Campo	Tipo	Restrição
id / userId	UUID	PK / FK User
name	varchar(80)	obrigatório
normalizedName	varchar(80)	índice único por usuário entre ativas, se viável no SQL
archivedAt	datetime UTC	nullable
createdAt / updatedAt	datetime UTC	obrigatório

StudyContent

Campo	Tipo	Restrição
id / userId / subjectId	UUID	PK / FK; subject pertence ao mesmo usuário
title	varchar(160)	obrigatório
notes	text	limite lógico de 10.000
materialUrl	text	nullable, protocolo http(s)
studiedOn	DATE	data civil informada ou padrão hoje local
reviewEnabled	boolean	padrão true; usuário escolhe
archivedAt	datetime UTC	nullable

Campo	Tipo	Restrição
createdAt / updatedAt	datetime UTC	obrigatório

ReturnPoint

Campo	Tipo	Restrição
id / userId	UUID	PK / FK
subjectId / contentId	UUID	opcionais; validar propriedade
context	text	até 2.000
nextAction	varchar(500)	obrigatório
materialUrl	text	opcional
status	enum	OPEN / DONE
createdAt / updatedAt	datetime UTC	obrigatório

FocusSession

Campo	Tipo	Restrição
id / userId	UUID	PK / FK
subjectId / contentId	UUID	opcionais, propriedade validada
plannedSeconds	int	300 a 7.200
activeSeconds	int	acumulado confirmado pelo servidor
status	enum	RUNNING, PAUSED, NEEDS_RECONCILIATION, COMPLETED, INTERRUPTED, DISCARDED
activeStartedAt	datetime UTC	nulo quando pausada/encerrada
startedAt / finishedAt	datetime UTC	datas de ciclo
version	int	controle concorrência / atualizações
createdAt / updatedAt	datetime UTC	obrigatório

Opcional FocusSegment (P1): segmentos {sessionId,startAt,endAt} para auditoria granular. Em P0, activeSeconds + activeStartedAt + histórico de transições estruturado já permitem cálculo, mas recomenda-se **FocusSegment** se o tempo permitir, pois melhora auditoria e testes.

ReviewSchedule

Campo	Tipo	Restrição
id / userId / contentId	UUID	PK / FKs
stepIndex	int	0 a 5
nextDueDate	DATE	data civil no fuso do usuário ao agendar
status	enum	ACTIVE / SUSPENDED
version	int	concorrência otimista
createdAt / updatedAt	datetime UTC	obrigatório

contentId é único em ReviewSchedule (um agendamento por conteúdo). Definir política de fuso: o vencimento é uma **data civil persistida**, não recalcular retroativamente ao trocar o fuso; apenas as próximas tentativas usam o novo fuso.

ReviewAttempt

Campo	Tipo	Restrição
id / userId / contentId / scheduleId	UUID	FKs
scheduleVersionBefore	int	versão de agendamento processada
rating	enum	AGAIN/HARD/GOOD/EASY
oldStepIndex / newStepIndex	int	auditoria
oldDueDate / newDueDate	DATE	auditoria
createdAt	datetime UTC	obrigatório
idempotencyKey	varchar(80)	única por usuário/ação

Índice único (scheduleId, scheduleVersionBefore) evita duas avaliações do mesmo ciclo. idempotencyKey adicional resolve retries por rede.

RewardEvent

Campo	Tipo	Restrição
id / userId	UUID	PK / FK
sourceType	enum	FOCUS_SESSION / REVIEW_ATTEMPT
sourceId	UUID	origem
xp	int	>= 0
localDate	DATE	data local do evento para limites
createdAt	datetime UTC	obrigatório

Índice único (sourceType,sourceId); XP total é soma de eventos. Restrições de teto diário devem ser avaliadas atomicamente com bloqueio por usuário/dia ou outra estratégia transacional equivalente, para impedir corrida de requisições. Não confiar somente em consulta seguida de create sem sincronização.

3.6 Relações e consistência

User 1--N AuthSession
User 1--N Subject 1--N StudyContent
User 1--N FocusSession
User 1--N ReturnPoint
StudyContent 1--0..1 ReviewSchedule 1--N ReviewAttempt
User 1--N RewardEvent

Todas as entidades privadas carregam userId para filtragem de propriedade. FKs isoladas não garantem propriedade cruzada automaticamente; validar subjectId / contentId com o usuário autenticado antes de associar. Em operações críticas, usar transações e índices compostos quando apropriado.

3.7 Índices sugeridos

- User.emailNormalized UNIQUE; AuthSession(userId,expiresAt).
- Subject(userId,archivedAt) e unicidade lógica de nome ativo por usuário.
- StudyContent(userId,subjectId,archivedAt), StudyContent(userId,studiedOn).

- ReturnPoint(userId,status,updatedAt DESC).
- FocusSession(userId,startedAt DESC), FocusSession(userId,status); unicidade parcial de sessão aberta por usuário via índice SQL parcial, se aplicável.
- ReviewSchedule(userId,status,nextDueDate) e ReviewSchedule(contentId) UNIQUE.
- ReviewAttempt(scheduleId,scheduleVersionBefore) UNIQUE.
- RewardEvent(sourceType,sourceId) UNIQUE, RewardEvent(userId,localDate,sourceType).

3.8 Segurança mínima obrigatória

1. **HTTPS em hospedagem**, cookies HttpOnly, Secure (produção), SameSite=Lax ou Strict, Path=/. Não guardar JWT em localStorage.
2. JWT de curta duração associado a AuthSession revogável; logout seta revokedAt e remove cookie. Se expirar, exigir novo login no MVP.
3. Proteger rotas de alteração por validação de Origin/Referer e estratégia CSRF adequada; SameSite sozinho não substitui CSRF. Manter implantação de produção preferencialmente em mesmo site.
4. Validar todos os DTOs (ValidationPipe com whitelist, forbidNonWhitelisted, transformação conforme necessidade); limites de comprimento e tamanho do payload.
5. helmet, rate limit para autenticação, CORS restrito a origem permitida e credentials: true quando necessário.
6. Hash de senha Argon2id; mensagens genéricas de credencial inválida; segredos fortes por env e nunca logs.
7. Verificar usuário em **cada** entidade, inclusive relacionamentos aninhados. Não confiar em userId recebido no body.
8. Erros de API sem stack trace ou hash de senha; logs com ID de requisição e sem notas de estudo em texto integral.
9. Renderizar notas como texto; se Markdown for adicionado futuramente, sanitizar. Links externos com rel="noopener noreferrer".
10. Separar dados de demonstração e dados reais; contas demo com credenciais de teste somente em ambiente local e nunca segredo embutido em produção.

3.9 Observabilidade e erros

Resposta de erro padrão:

```
{
  "error": {
    "code": "FOCUS_INVALID_TRANSITION",
    "message": "Esta sessão não pode ser retomada.",
    "details": {},
    "requestId": "req-demo-123"
  }
}
```

Códigos típicos: 400 entrada inválida; 401 sessão não autenticada; 403 não autorizado; 404 não encontrado (preferencial para ocultar recursos de outros usuários); 409 estado/versão incompatível; 422 regra de negócio; 429 rate-limit; 500 erro inesperado. Incluir `/api/v1/health` sem informações sensíveis.

3.10 Configuração e execução

Variáveis de exemplo: `DATABASE_URL`, `JWT_SECRET`, `FRONTEND_ORIGIN`, `COOKIE_SECURE`, `PORT`, `NODE_ENV`. Usar `apps/api/.env.example` com placeholders, jamais senhas funcionais. `docker-compose.yml` fornece somente banco Postgres com volume persistente para desenvolvimento. Seed é idempotente, usa dados falsos e deve exigir comando explícito.

Prisma 7: utilizar `prisma.config.ts`, `@prisma/client@7`, `prisma@7`, `@prisma/adapter-pg` e geração de client atual. Checar requisitos de Node/TypeScript no momento da instalação. Não copiar APIs antigas como `new PrismaClient()` sem adapter.

3.11 Estratégia de deploy

- Desenvolvimento: Vite e NestJS locais; PostgreSQL no Docker.
- Demonstração: executar localmente em máquina preparada ou hospedar SPA/API/Postgres em serviços apropriados, sem promessa de gratuidade permanente.
- Produção pública futura: HTTPS, backup de banco, política de retenção, recuperação de senha, monitoramento e direitos do titular de dados antes de abertura geral.

3.12 Fontes técnicas de referência

- NestJS: https://docs.nestjs.com/
- NestJS + Prisma: https://docs.nestjs.com/data/prisma
- Prisma v7 NestJS: https://www.prisma.io/docs/guides/v7/frameworks/nestjs
- Prisma ORM v7: https://www.prisma.io/docs/orm/v7
- Vite: https://vite.dev/guide/
- OWASP Session Management: https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet

04 — CONTRATOS DE API, ESTADOS E ALGORITMOS

4.1 Convenções REST

Base: `/api/v1`. Endpoints privados exigem autenticação. Todas as datas de instantes são ISO 8601 UTC; vencimentos de revisão são strings `YYYY-MM-DD` no calendário civil do usuário. IDs UUID. Paginação `{items, page, pageSize, total}` para listas grandes; padrão `pageSize=20`, teto 100. Endpoints de escrita aceitam JSON e validam DTOs. Campos geridos pelo servidor (`userId`, `xp`, `createdAt`, `activeSeconds`) nunca são confiados ao body do cliente.

Idempotência: ações de conclusão aceitam header `Idempotency-Key` UUID; o servidor persiste a chave no evento/registro correspondente e retorna a mesma operação em repetição. Para transições incompatíveis com nova chave, retornar 409. Excluir metadados de autenticação de logs.

4.2 Catálogo de endpoints P0

Método	Caminho	Propósito	Sucesso
POST	/auth/register	Criar usuário e iniciar	201

Método	Caminho	Propósito	Sucesso
		sessão	
POST	/auth/login	Login	200
POST	/auth/logout	Revogar sessão	204
GET	/auth/me	Perfil autenticado	200
PATCH	/users/me	Nome e fuso	200
GET	/subjects	Listar disciplinas	200
POST	/subjects	Criar disciplina	201
PATCH	/subjects/:id	Renomear/arquivar	200
GET	/contents	Listar/pesquisar conteúdos	200
POST	/contents	Criar conteúdo livre	201
GET	/contents/:id	Detalhar conteúdo	200
PATCH	/contents/:id	Editar/arquivar/habilitar revisão	200
POST	/focus-sessions	Iniciar sessão	201
GET	/focus-sessions/active	Recuperar sessão aberta ou null	200
GET	/focus-sessions	Histórico paginado	200
POST	/focus-sessions/:id/pause	Pausar	200
POST	/focus-sessions/:id/resume	Retomar	200
POST	/focus-sessions/:id/finish	Concluir/interromper	200
POST	/focus-sessions/:id/reconcile	Resolver sessão pendente	200
GET	/return-points	Listar pontos de retorno	200
POST	/return-points	Criar ponto	201
PATCH	/return-points/:id	Editar/concluir	200
GET	/reviews/due	Revisões vencidas/hoje	200
GET	/reviews/upcoming	Próximas revisões	200
POST	/reviews/:scheduleId/attempts	Avaliar revisão e reagendar	201
GET	/reviews/:scheduleId/history	Histórico de avaliações	200
GET	/academy	XP, estágio, assets	200
GET	/statistics/summary	Métricas filtradas	200
GET	/dashboard	Visão agregada em uma chamada	200
GET	/health	Estado mínimo da API	200

Endpoints adicionais de delete físico, reset de XP, ranking e anexos não pertencem a P0.

4.3 Exemplos de contratos JSON

POST /contents

```
{
  "subjectId": "00000000-0000-4000-8000-000000000001",
  "title": "Herança em Java",
  "notes": "Conceito de herança e sobrescrita de métodos.",
  "materialUrl": "https://example.org/aula/18",
```

```
  "studiedOn": "2026-11-02",
  "reviewEnabled": true
}
```


Resposta 201 (exemplo):


```
{
  "id": "00000000-0000-4000-8000-000000000002",
  "title": "Herança em Java",
  "review": {"status":"ACTIVE", "stepIndex":0, "nextDueDate":"2026-11-03"}
}
```


POST /focus-sessions


```
{"plannedMinutes":25,"subjectId":null,"contentId":null}
```


Resposta 201:


```
{
  "id":"00000000-0000-4000-8000-000000000003",
  "status":"RUNNING","plannedSeconds":1500,
  "activeSeconds":0,"activeStartedAt":"2026-11-02T13:00:00Z",
  "serverNow":"2026-11-02T13:00:00Z","version":1
}
```


POST /focus-sessions/:id/finish


```
{"outcome":"COMPLETED"}
```


Retornar activeSeconds, xpGranted, totalXp e status; não aceitar xp calculado no cliente. Para INTERRUPTED, permitir tempo parcial válido. Endpoint deve ser idempotente.

POST /focus-sessions/:id/reconcile


```
{"outcome":"INTERRUPTED","confirmedActiveSeconds":600}
```


confirmedActiveSeconds é autodeclarado em recuperação e deve ser validado contra um teto plausível inferido de timestamps de servidor e duração planejada. Não confundir com verificação real de foco. DISCARDED não concede XP.

POST /return-points


```
{
  "subjectId": null,
  "contentId": null,
  "context":"A aula parou no exemplo de sobrescrita.",
  "nextAction":"Abrir o projeto e executar o exemplo com duas subclasses.",
  "materialUrl":"https://example.org/aula/18"
}
```

POST /reviews/:scheduleId/attempts

{"rating":"GOOD","expectedVersion":2}

Resposta 201:

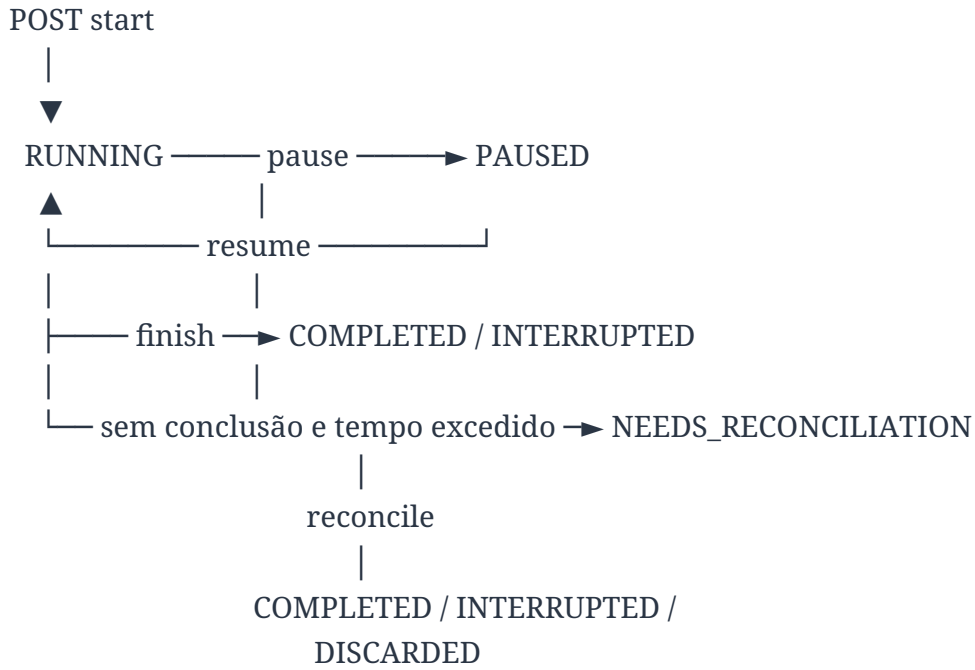
{
 "attemptId":"00000000-0000-4000-8000-000000000004",
 "oldStepIndex":1,"newStepIndex":2,
 "nextDueDate":"2026-11-09","xpGranted":5
}

Se a versão não corresponder, devolver 409 com status atual para o cliente atualizar; nunca registrar duas revisões concorrentes do mesmo agendamento.

GET /dashboard (exemplo parcial)

{
 "focusTodaySeconds":1500,
 "dueReviewCount":2,
 "activeSession":null,
 "nextReturnPoint":{"id":"...","nextAction":"..."},
 "academy":{"totalXp":355,"stage":3}
}

4.4 Máquina de estados de foco



Transições terminais são imutáveis; não há pause/resume após finalizar. O usuário não pode ter duas sessões abertas simultaneamente. A API deve ser resiliente a requisições concorrentes e retornos tardios.

Cálculo de duração

```
on start:    status=RUNNING; activeStartedAt=serverNow; activeSeconds=0
on pause:    activeSeconds += clamp(now - activeStartedAt, 0, remainingPlanned)
             activeStartedAt=null; status=PAUSED
on resume:   activeStartedAt=serverNow; status=RUNNING
on finish:   if RUNNING, consolidar segmento até now
             eligibleSeconds=min(activeSeconds, plannedSeconds)
             se conclusão válida, calcular XP e gravar evento na mesma transação
```

Teto por sessão: segmentos nunca somam mais que a duração planejada. O timer no frontend pode mostrar `max(0, plannedSeconds - eligibleSeconds - currentElapsed)` e não depende de `setInterval` para *calcular* a verdade. `setInterval` serve apenas para repintar a UI.

Caso sem rede/aba fechada: o backend não consegue certificar se o usuário continuou estudando. Se não houver finalização e passar `plannedSeconds + 900s` desde o início ativo (com consideração de pausas), marcar como pendente na próxima leitura e solicitar confirmação humana. Não conceder XP sem decisão explícita.

Recompensa por foco

```
eligibleMinutes = floor(eligibleSeconds / 60)
rawXp = eligibleMinutes < 5 ? 0 : min(eligibleMinutes, 60)
alreadyAwardedToday = soma eventos FOCUS_SESSION localDate=hojeLocal
xpGranted = min(rawXp, max(0, 180 - alreadyAwardedToday))
```

Para interrupção com 12 minutos de foco efetivo, o evento concede até 12 XP; 4 minutos concedem 0. A UI nunca deve dizer que XP prova qualidade da concentração. Usar transação serializável/lock por usuário + data de recompensa e restrições únicas para evitar corrida entre duas finalizações.

4.5 Algoritmo de revisão inicial

Não se trata de algoritmo científico ideal. É uma **heurística transparente de repetição espaçada**, adequada ao MVP universal sem conteúdos predefinidos.

Configuração P0: `intervals = [1, 3, 7, 14, 30, 60]` (dias civis). Cada conteúdo tem `stepIndex`. Ao ativar revisão, índice 0 e primeiro vencimento amanhã na data local. No vencimento, o usuário realiza recuperação ativa e autoavalia.

```ts
const INTERVALS_DAYS = [1, 3, 7, 14, 30, 60] as const;

type ReviewRating = 'AGAIN' | 'HARD' | 'GOOD' | 'EASY';

function nextStep(current: number, rating: ReviewRating): number {
 switch (rating) {
  case 'AGAIN': return 0;
  case 'HARD':  return current;
  case 'GOOD':  return Math.min(current + 1, 5);
  case 'EASY':  return Math.min(current + 2, 5);
 }
```

```
}

// nextDueDate = adicionarDiasCivis(dataLocalDaResposta,
//                                  INTERVALS_DAYS[nextStep(...)])
// Não usar soma ingênua de 86.400.000 ms para datas civis com DST.
```

Exemplos de tabela de decisão:

Índice anterior	Avaliação	Novo índice	Próximo vencimento
0 (1 dia)	AGAIN	0	+1 dia
0 (1 dia)	HARD	0	+1 dia
0 (1 dia)	GOOD	1	+3 dias
1 (3 dias)	GOOD	2	+7 dias
2 (7 dias)	EASY	4	+30 dias
4 (30 dias)	HARD	4	+30 dias
5 (60 dias)	GOOD	5	+60 dias

Ponto importante: o atraso não multiplica revisões vencidas. Se a revisão estava marcada para 01/11 e foi concluída em 10/11, contar o próximo intervalo desde **10/11**. Guardar histórico de data anterior e nova data.

Recompensas de revisão: uma conclusão válida registra `ReviewAttempt` e `RewardEvent` de até 5 XP, sujeito ao limite de 50 XP/dia. Registrar avaliação + novo vencimento + recompensa numa transação. A tela de revisão sempre mostra conteúdo/título e permite tentar lembrar **antes** de revelar as notas.

Fuso e datas: usar biblioteca compatível com IANA para `localDate` e aritmética de calendário. Persistir `nextDueDate` como DATE; consultar com `nextDueDate <= hojeLocal`. Horário do dia é irrelevante em P0 (sem lembrete agendado externo).

4.6 Academia e progressão

```
function academyStage(totalXp: number): 1 | 2 | 3 | 4 {
  if (totalXp >= 800) return 4;
  if (totalXp >= 350) return 3;
  if (totalXp >= 100) return 2;
  return 1;
}
```

A API envia `{totalXp, focusXp, reviewXp, stage, stageStartXp, nextStageXp}`. Estágio IV não possui próximo limiar (`nextStageXp=null`). Frontend usa quatro ilustrações SVG pré-definidas e uma animação curta opcional apenas **após** sessão/revisão, nunca durante o foco.

4.7 Métricas e painéis

- `focusSecondsByPeriod`: somatório `eligibleSeconds` de sessões concluídas/interrompidas, filtrado por UTC e agrupado por data local de conclusão.
- `reviewCount`: número de ReviewAttempt válidas, não de cartões abertos.
- `dueReviewCount`: número de agendamentos ativos com `nextDueDate <= todayLocal`.
- `daysWithStudy`: dias locais distintos com foco >= 5 minutos registrado; não usar streak punitivo.

- `totalXp`: soma de RewardEvent; nunca computed client-side para persistência.
- Não criar métrica “porcentagem de domínio”, “nível de atenção” nem “retenção comprovada” no P0.

4.8 Casos limite obrigatórios

1. Duas abas tentam iniciar foco: uma ganha; a outra recebe 409 com sessão ativa.
2. Reenvio da conclusão após timeout: mesma resposta, XP invariável.
3. Duas revisões simultâneas de um conteúdo: uma registra, outra recebe 409.
4. Conteúdo com notas vazias: pode ser revisado por lembrança do tema/título.
5. Usuário troca fuso: datas antigas de revisão permanecem como datas civis registradas; novos vencimentos usam fuso atual.
6. Relógio do cliente adiantado: eventos são datados pelo servidor.
7. Usuário arquiva disciplina com conteúdos: manter histórico; conteúdos ficam ocultos por padrão; não excluir em cascata por engano.
8. Link de material é `javascript:`: rejeitar 400.
9. Sessão com pausa antes de completar 5 minutos: registro válido, sem XP.
10. Fila com 50 revisões atrasadas: cada conteúdo aparece uma vez; UI pode paginar ou agrupar.

05 — EXPERIÊNCIA DO USUÁRIO, FLUXOS E ACADEMIA VIVA

5.1 Princípio orientador

O aplicativo precisa facilitar a entrada e a continuidade do estudo sem competir pela atenção durante o trabalho. A gamificação aparece antes/depois, não durante o foco. Não são permitidos pop-ups de XP, chuva de confetes, notificações intrusivas ou telas de gestão da academia sobre a atividade principal.

A aplicação acompanha materiais **externos** e **quaisquer disciplinas**, sem exigir exercícios, trilhas ou correção de conteúdo próprios. O usuário controla o que estuda e descreve o que aprendeu; o sistema controla dados, tempo registrado, agenda e recompensas.

5.2 Navegação principal (mobile e desktop)

Tela / rota	Ação principal	Estados a projetar
Acesso /login, /register	Entrar/criar conta	carregando, erro genérico, validação, autenticação expirou
Inicial /app	**Continuar estudando** ou iniciar foco	sem disciplina, ponto de retorno, foco pendente, revisões vencidas, academia
Foco /app/focus	Iniciar, pausar, retomar, concluir	pronto, RUNNING, PAUSED, reconciliação, concluído, offline/erro
Disciplinas /app/subjects	Criar/renomear/arquivar	vazio, lista, edição, arquivamento
Conteúdos /app/contents	Registrar assunto estudado	lista, novo, detalhe, edição, revisões habilitadas/desabilitadas
Revisões /app/reviews	Tentar recordar um conteúdo	vencida, futura, texto oculto/revelado, avaliação, concluída
Continuidade /app/continue	Abrir ponto de retorno	sem ponto, ponto ativo, histórico, marcar concluído
Academia /app/academy	Ver evolução	quatro estágios, falta XP para próximo, limite máximo

Tela / rota	Ação principal	Estados a projetar
Estatísticas /app/statistics	Entender a rotina	vazia, período selecionado, métricas por matéria
Perfil /app/settings	Editar nome e fuso	confirmação e aviso sobre dias civis

As URLs são propostas; a nomenclatura exata pode ser ajustada sem alterar as regras.

5.3 Dashboard e hierarquia de informação

Ordem sugerida de prioridade:

1. Se houver sessão pendente de reconciliação, destacar resolução (sem XP automático).
2. Se houver sessão RUNNING/PAUSED, oferecer retorno direto ao timer.
3. Se existir ponto de retorno aberto, destacar “Continue de onde parou”, com **próxima ação**.
4. Caso contrário, destacar CTA “Começar uma sessão de foco”.
5. Exibir revisões vencidas/hoje em seção acessível, mas **não bloquear** o estudo por atraso.
6. Exibir resumo de tempo registrado e uma prévia discreta da academia.

Estado vazio: “Você pode começar a estudar agora; nenhuma disciplina ou conteúdo precisa ser cadastrado antes.” Dar opção de criar disciplina, nunca impor formulário inicial longo.

5.4 Fluxo de foco — detalhamento UX

1. Usuário escolhe 5–120 minutos (padrões 10/25/50) e, opcionalmente, vincula uma disciplina ou conteúdo.
2. Clica **Iniciar foco**; frontend aguarda confirmação do backend.
3. A tela exibe tempo restante, nome da disciplina opcional e botões “Pausar” e “Encerrar”. Link discreto “Onde parei?” só se necessário, nunca caixa de diálogo recorrente.
4. **Pausar:** contador visual congela; backend registra o tempo até a pausa.
5. **Retomar:** backend inicia novo segmento. O cronômetro renderiza a partir dos dados do servidor, sem confiar no relógio local como verdade do saldo.
6. **Concluir/interromper:** interface pergunta/indica o resultado e aguarda confirmação. Encerramento abaixo de 5 minutos é válido, mas não gera XP.
7. Resumo pós-foco: minutos efetivos registrados, XP concedido (inclusive zero por limite diário), novo estágio se aplicável e botões opcionais **Registrar conteúdo** / **Salvar ponto de retorno** / **Voltar ao início**.
8. Se o usuário omitir os dois formulários, a sessão ainda permanece registrada e a recompensa é preservada.

Falha de conexão ou fechamento da aba: não simular XP. Ao voltar, recuperar estado do servidor; sessões ultrapassadas exigem reconciliação. Se não houver backend acessível, mostrar erro objetivo e oferecer nova tentativa, sem registrar sucesso falso.

5.5 Fluxo de registro de conteúdos — sem fricção

Formulário P0:

- Disciplina: obrigatória (já escolhida quando o conteúdo foi iniciado a partir de uma disciplina); pode ser criada em diálogo leve.
- Título: obrigatório; sugestão do próprio usuário, nenhum catálogo.
- Notas: opcionais, texto puro, 10.000 caracteres.
- Link do material: opcional, somente HTTP(S).

- Data estudada: hoje no fuso do perfil por padrão, pode editar.
- “Quero revisar periodicamente este conteúdo”: opção explícita, ligada por padrão apenas se a interface explicar a consequência; o usuário pode desligar.

Não pedir resumo longo, anexos, categoria, questões nem nota de desempenho no primeiro registro. O objetivo é salvar **um lembrete revivível** e iniciar a agenda de revisão.

5.6 Fluxo de revisão — recuperação antes de revelar

1. Fila mostra título, disciplina, data de vencimento e dias de atraso (sem mensagens de culpa).
2. Ao abrir, exibir título e convite: “O que você consegue lembrar sobre esse assunto?”
3. **Notas do usuário começam ocultas**. Link externo permanece disponível, mas mostrar orientação para tentar lembrar antes de abrir.
4. Botão “Ver minhas anotações” revela texto, se houver; se não houver, a revisão ainda pode ser concluída.
5. Escolher uma autoavaliação: **Não lembrei**, **Com dificuldade**, **Lembrei bem**, **Muito fácil**.
6. Frontend envia `rating` + chave idempotente + versão esperada; servidor grava tentativa, próxima data e XP de forma atômica.
7. Exibir nova data e lembrete de que o resultado é **autoavaliação**, não correção objetiva.

Revisões atrasadas: uma unidade por conteúdo, sem multiplicar cartões por todos os dias perdidos. Não forçar o estudante a “pagar a dívida” antes de iniciar novas atividades.

5.7 Fluxo de continuidade — memória externa

Campos: disciplina e conteúdo opcionais, **onde parei** (opcional), **próxima ação** (obrigatória), URL (opcional). O usuário pode criar, editar, finalizar e manter vários pontos. O dashboard destaca um ponto aberto mais recentemente atualizado.

Exemplo de microcópia:

> **Onde você parou:** a aula terminou no exemplo de classes abstratas.
> **Próximo passo:** abrir o projeto e executar o exemplo com duas subclasses.
> **Material:** abrir link salvo.

A próxima ação é concreta e executável. Se o usuário não souber preenchê-la, exibir exemplos curtos, **sem gerar conteúdo por IA**.

5.8 Academia Viva — design do sistema decorativo

Papel: celebração visual do progresso. Não é jogo de administração, não possui moedas, inventário, compras, arrastar prédios, produção de recursos ou personagens simulados em P0.

Estágio	XP cumulativo	Visual conceitual	Regra
I — Fundação	0–99	terreno, edifício inicial e jardim simples	padrão de todos os usuários
II — Campus	100–349	biblioteca e caminhos	desbloqueio automático
III — Universidade	350–799	laboratórios e ampliação de jardins	desbloqueio automático
IV — Cidade do Conhecimento	>= 800	conjunto acadêmico amplo	último estágio P0

Apresentação técnica: um SVG criado no próprio projeto para cada estágio, renderizado no frontend conforme `stage` vindo da API. Se o usuário atravessar vários limiares de uma vez, mostrar diretamente estágio correto; não tentar reexecutar uma sequência de efeitos. As ilustrações não podem conter marcas ou personagens protegidos por direitos autorais sem licença.

Recompensas: foco e revisão geram XP. A academia não distingue verdade de conhecimento adquirido; a mensagem deve ser "Sua academia evoluiu com suas sessões e revisões registradas", e não "Sua inteligência aumentou".

5.9 Gamificação responsável

- Sem streak diário obrigatório, punição por ausência, perda de XP ou degradação de edifícios.
- Sem recompensa por abrir cartão de revisão sem concluir avaliação.
- Sem chamar foco registrado de atenção neural medida.
- Recompensa de foco inclusive quando foi interrompido, proporcional ao tempo válido, sujeito a mínimo/tetos.
- Melhorias visuais aparecem **depois** do estudo; tela de foco é intencionalmente simples.
- Exibir quando o limite de XP diário foi alcançado, sem esconder o tempo estudado.
- Priorizar progresso individual, sem ranking social.

5.10 P1 — assistência de início e missões por tamanho

Assistência "Não consigo começar" (não bloquear o MVP): usuário escolhe entre "não sei o que fazer", "parece grande demais", "tenho receio de errar" e "estou cansado/confuso". O app apresenta uma sugestão curta e editável, sem diagnóstico ou técnica infalível.

Três modos de missão: essencial (ação mínima), completo (estudo + revisão), profundo (aplicação com material externo). Devem ser apenas modelos de planejamento, não um currículo fechado. Não exigir prova criada pelo aplicativo.

5.11 Design visual e acessibilidade

- Interface em português brasileiro, textos legíveis e layout mobile-first.
- Paleta sugerida: azul-noite para estrutura, turquesa/verde suave para progresso e neutros claros/escuros; evitar excesso de cor durante foco.
- Tipografia simples e acessível; contraste mínimo compatível com WCAG AA nos componentes essenciais.
- Foco de teclado visível; botões com rótulos; `aria-live` discreto para estados importantes; não anunciar cada segundo do timer a leitores de tela.
- Preferência `prefers-reduced-motion`: desativar transições de recompensa e animações da academia.
- Carregamento com esqueleto/placeholder; erros com opção de tentar novamente.
- Não esconder funções essenciais atrás de hover ou efeitos visuais.

5.12 Mensagens e comunicação

Correto: "Você registrou 25 minutos de estudo", "Sua próxima revisão está marcada para 5 de novembro", "Você retomou uma atividade que havia deixado preparada".

Evitar: "Sua atenção melhorou 30%", "Você dominou definitivamente este tema", "Falhou por perder a sequência", "Seu cérebro foi recuperado".

5.13 Fluxos de aceitação visual

- Sem conta: visitante vê cadastro/login e nenhuma informação privada.
- Conta vazia: usuário inicia foco avulso sem primeiro criar disciplina.
- Conteúdo sem notas: revisão oferece recordação do título e avaliação.
- Sessão já ativa: usuário é encaminhado ao estado existente, não cria outra.
- Sessão pendente: resolver/discartar antes de iniciar nova (sem XP fabricado).
- Revisões atrasadas: fila continua utilizável e não exibe cobranças repetitivas.
- Usuário no estágio IV: não mostrar barra impossível de completar para estágio V.

06 — ESTRATÉGIA DE TESTES, QUALIDADE E CRITÉRIOS DE CONCLUSÃO

6.1 Objetivo

O projeto precisa ser **demonstrável, reprodutível e verificável**, não apenas renderizar telas. A qualidade mínima se concentra em **autenticação**, **isolamento de dados**, **máquinas de estado**, **cálculo de tempo e XP**, **agenda de revisões**, **reconciliação**, **continuidade**, **integridade transacional** e **migrações**.

6.2 Pirâmide de testes

Camada	Ferramentas sugeridas	O que testar	Prioridade
Unitária do domínio	Jest no backend	intervalos, XP, estágios, transições, data local	P0
Integração API/banco	Jest + Supertest + PostgreSQL de teste	CRUD, constraints, propriedade, transações	P0
Componentes React	Vitest + React Testing Library	formulário, timer, estados e acessibilidade básica	P0 essencial
Fluxos de ponta a ponta	Playwright (opcional)	cadastro → foco → conteúdo → revisão → academia	P1 se tempo
Manual	navegador desktop e mobile	UX, recarga, perda de rede, abas, foco, correções	P0

Testes de tempo e revisão devem usar **relógio injetado/congelado** (ClockService e fuso controlado), não sleep() de 25 minutos. O backend deve permitir teste sem conceder privilégios especiais em produção.

6.3 Matriz crítica — foco

ID	Cenário	Resultado esperado
TF-01	Foco de 25min realizado, sem pausa	contabiliza <= 1.500 segundos; XP de acordo com regra
TF-02	Foco 5min + pausa 2min + 5min	conta 10min, não 12min
TF-03	Interrupção após 4min	salva sessão, XP=0
TF-04	Interrupção após 12min	XP<=12, limitado por regras
TF-05	Foco 90min	sessão pode existir, XP<=60
TF-06	limite diário de foco=180	próximos eventos geram XP=0; tempo continua registrado
TF-07	POST finish repetido	não duplica nem XP nem conclusão

ID	Cenário	Resultado esperado
TF-08	duas sessões simultâneas do mesmo usuário	somente uma sessão aberta
TF-09	duas chamadas concorrentes de finish	apenas uma transação gera evento
TF-10	refresh da página	exibe estado restaurado do backend
TF-11	logout durante foco	sem prêmio automático; sessão persiste com segurança
TF-12	aba fechada e duração excedida	NEEDS_RECONCILIATION, sem XP até confirmação
TF-13	confirmedActiveSeconds negativo/acima do teto	requisição rejeitada
TF-14	clock local alterado	backend mantém duração autorizada
TF-15	cliente envia XP forjado	backend rejeita/ignora campos não previstos

6.4 Matriz crítica — revisões

ID	Cenário	Resultado esperado
TR-01	cadastrar conteúdo com revisão	agenda 1 dia civil depois da data de habilitação local
TR-02	cadastrar sem revisão	sem agendamento ativo
TR-03	índice=2, AGAIN	novo índice=0, próxima +1 dia
TR-04	índice=2, HARD	mantém índice=2, próxima +7 dias
TR-05	índice=2, GOOD	índice=3, próxima +14 dias
TR-06	índice=2, EASY	índice=4, próxima +30 dias
TR-07	índice=5, EASY	permanece 5, próxima +60 dias
TR-08	revisão atrasada 10 dias	uma única pendência; cálculo contado do dia realizado
TR-09	requisição de revisão repetida	mesma tentativa, sem XP adicional
TR-10	revisão concorrente com versão antiga	conflito 409 e nenhuma tentativa duplicada
TR-11	limite diário XP revisão=50	tentativas persistem, XP extra=0
TR-12	desabilitar revisão	não é listada na fila, histórico permanece
TR-13	arquivar conteúdo	some da fila ativa, sem cascade destrutivo
TR-14	DST / meia-noite com fuso válido	aritmética de dias civis correta, sem somar 24h cegamente
TR-15	conteúdo sem notas	pode completar recuperação pelo título/material externo

6.5 Matriz — segurança e consistência

ID	Cenário	Resultado esperado
TS-01	usuário B acessa conteúdo ID de A	404/403 sem dados de A
TS-02	usuário B aponta subjectId de A em conteúdo novo	rejeição de vínculo
TS-03	e-mail duplicado	erro controlado, sem criação parcial
TS-04	senha inválida	erro genérico, sem revelar existência do usuário

ID	Cenário	Resultado esperado
TS-05	logout e reutilização do mesmo cookie	sessão revogada; 401
TS-06	POST cross-origin malicioso com cookie	rejeição por defesa CSRF/Origin
TS-07	usuário manda HTML/script nas notas	exibido como texto, nunca executado
TS-08	link javascript: em material	400; não é salvo
TS-09	relatório de outra conta	não vaza estatísticas/XP
TS-10	falha entre atualização de revisão e recompensa	transação rollback, estado consistente
TS-11	migração em banco vazio	cria todas as tabelas e índices
TS-12	seed executado duas vezes	não duplica demonstração nem altera usuário real

6.6 Acessibilidade e dispositivos

- Navegação básica por teclado nos modais/formulários.
- Rótulos legíveis, feedback de erro e contraste suficiente.
- Formulários navegáveis em largura de **360px**, tablet e desktop.
- Cronômetro usa valores textuais estáveis; evitar anúncios de segundo a segundo a leitores de tela.
- Com prefers-reduced-motion, nenhuma animação é essencial para compreender a recompensa.
- Não depender de hover para editar conteúdos ou iniciar revisão.

6.7 Dados de demonstração e cenários controlados

Gerar via comando explícito uma conta **fictícia local**, com duas disciplinas (ex.: História e Inglês), conteúdos curtos inventados, um ponto de retorno e revisões em estados conhecidos. Nenhuma credencial real, dado corporativo, identificador pessoal, screenshot de sistema interno ou registro confidencial.

Para demonstrar mudanças de estágio e revisões vencidas, **não** manipular tempo real nem forjar XP por API pública. Usar fixtures/seed controlado em ambiente local de demo, com marcação visual de que o conteúdo é fictício e comando dedicado. Essa seed não deve ser habilitada em produção.

6.8 DoD — Definition of Done por funcionalidade

Uma tarefa está **concluída** somente se:

1. Requisitos e regra(s) vinculadas estão identificados.
2. Funciona na UI e/ou API em um caminho feliz e um de erro.
3. Backend valida DTO, autenticação e propriedade do recurso.
4. Operações críticas são transacionais/idempotentes e cobertas por teste.
5. Frontend trata espera, erro, vazio e sucesso confirmado, sem inventar persistência.
6. Não há alteração não documentada nas regras de XP, revisão ou estágio.
7. lint, typecheck, testes pertinentes e build passam.
8. A feature foi exercitada manualmente após refresh/relogin.
9. O Codex registrou alterações, comando de teste, evidência e limitações em docs/evidencias-ia/.

6.9 DoD — aplicação pronta para apresentação

- ☐ `docker compose up -d` sobe banco em máquina preparada.
- ☐ migrações Prisma são aplicadas com instrução documentada.
- ☐ API sobe com variáveis locais seguras e `/api/v1/health` funciona.
- ☐ SPA sobe, autentica, navega e mantém dados após refresh/relogin.
- ☐ CRUD de disciplinas/conteúdos registra material de qualquer área.
- ☐ Foco avulso inicia, pausa, retoma, conclui e recompensa sem duplicidade.
- ☐ Sessão pendente pode ser conciliada ou descartada.
- ☐ Ponto de retorno reaparece após novo login.
- ☐ Revisão aparece agendada, admite autoavaliação e reagenda.
- ☐ XP e academia permanecem coerentes com eventos armazenados.
- ☐ Testes centrais e build foram executados; relatórios incluídos.
- ☐ Dados de demo são fictícios; segredos e registros privados estão ocultos.
- ☐ Existe vídeo narrado mostrando o fluxo funcionando.
- ☐ Evidências do uso de IA durante desenvolvimento são rastreáveis.

6.10 Métricas do produto vs. promessas

Pode afirmar: quantidade de sessões registradas, tempo contabilizado, conteúdos cadastrados, revisões concluídas, dias com estudo, retomadas, XP e estágio visual.

Não pode afirmar com base no MVP: nível cerebral de atenção, diagnóstico de TDAH, domínio comprovado de matéria arbitrária, aumento causal de retenção, aprendizado certificado ou medida objetiva de foco.

07 — PLANO DE EXECUÇÃO COM CODEX E CONTROLE DE ESCOPO

7.1 Estratégia

Uma linguagem de ponta a ponta, mas com backend real: TypeScript no React e no NestJS, banco PostgreSQL e Prisma. Implementar um **monólito modular** e um ciclo vertical completo antes de detalhes cosméticos. A academia é decorativa e não possui engine de jogos.

O plano foi desenhado para reduzir retrabalho e custo de contexto do agente. **Não é garantia de desenvolvimento integral em um dia.** A arquitetura deve estar correta, enquanto o escopo visual e as melhorias P1 podem esperar.

7.2 Ordem de leitura obrigatória do Codex

1. `docs/00_LEIA_PRIMEIRO.md`.
2. `docs/01_PRODUTO_ESCOPO.md` e `docs/02_REQUISITOS_REGRAS.md`.
3. `docs/03_ARQUITETURA_DADOS.md` e `docs/04_API_ALGORITMOS.md`.
4. `docs/05_UX_GAMIFICACAO.md` e `docs/06_TESTES_QUALIDADE.md`.
5. `docs/07_PLANO_CODEX.md` e `AGENTS.md`.

Ordem de precedência em conflito: `02_REQUISITOS_REGRAS` > `04_API_ALGORITMOS` > `03_ARQUITETURA_DADOS` > `05_UX_GAMIFICACAO` > `01_PRODUTO_ESCOPO`; se conflito material permanecer, registrar e não inventar comportamento silenciosamente.

7.3 Sequência de implementação com entregas verificáveis

Etapa 0 — Base reprodutível

Construir: monorepo npm workspaces, apps web/api, configuração TypeScript strict, Tailwind, Nest, Prisma 7 + adapter, PostgreSQL Compose, .env.example, health check, scripts dev/test/build/lint/typecheck.

Aceitar se: instruções de README funcionam em máquina nova; banco migra; API responde; SPA abre; nenhuma credencial no Git.

Etapa 1 — Autenticação + disciplinas

Construir: usuário, AuthSession, hash Argon2id, cookie, login/logout, guards, validação, rotas privadas, CRUD de disciplinas com arquivamento, teste de isolamento entre contas.

Aceitar se: cadastro e login persistem; logout revoga sessão; B não acessa disciplinas de A.

Etapa 2 — Foco robusto + recompensas

Construir: máquina de estados, timer reconstituído pelo servidor, pausa e retomada, encerramento, reconciliação, RewardEvent e tetos diários. Tela de foco sem distrações.

Aceitar se: refresh não zera timer; POST repetido não concede XP novo; pausa não conta; sessão pendente não gera XP automático.

Etapa 3 — Conteúdos + continuidade

Construir: CRUD de assuntos registrados manualmente, link/nota, revisão habilitada, ReturnPoint independente, dashboard com próximo passo, telas de vazio.

Aceitar se: o usuário registra História, Java ou qualquer outra área sem catálogo, não precisa cadastrar um exercício, salva onde parou e recupera após login.

Etapa 4 — Revisões periódicas

Construir: ReviewSchedule, ReviewAttempt, algoritmo de intervalos de dias civis, autoavaliação, fila vencida/futura, transação com XP, controles de versão/idempotência.

Aceitar se: primeira revisão é amanhã (fuso local), atraso não se acumula em N tarefas, rating altera data, reenvio não pontua duas vezes.

Etapa 5 — Academia e estatísticas

Construir: stage derivado de XP, quatro SVGS próprios (podem ser estilizados sem alta complexidade), preview e tela academia, métricas por período.

Aceitar se: atingir XP limiar muda estágio; falta de uso não reduz progresso; painel só exibe fatos registrados.

Etapa 6 — Polimento + apresentação

Construir: tratamento uniforme de erro, responsividade, acessibilidade básica, seed de demo isolada, documentação de execução, evidências GenAI, roteiro e gravação narrada.

Aceitar se: DoD em 06 e roteiro em 08 passam; projeto executa sem depender de API de tokens.

7.4 Estratégia caso exista apenas um dia

Regra: concluir e testar o conjunto de P0 antes de P1. Se o prazo encurtar:

1. Não remover backend, banco, autenticação nem testes dos invariantes críticos.
2. Reduzir primeiro os efeitos visuais (quatro SVGs simples, sem animação).
3. Simplificar estatísticas para duas métricas confiáveis e expansão futura.
4. Priorizar foco, conteúdo, ponto de retorno e uma revisão ponta a ponta.
5. Adiar filtros avançados, animações, personalização, exportações e diagnósticos de início.
6. Reportar honestamente toda funcionalidade P0 incompleta; não marcar telas estáticas como concluídas.

7.5 Templates de prompt para Codex

Prompt de abertura

> Leia AGENTS.md e a documentação em docs/, em especial 02_REQUISITOS_REGRAS.md. Não programe antes de resumir em até 20 linhas o escopo P0, a stack, os invariantes críticos e a primeira etapa. Implemente SOMENTE a Etapa 0 do 07_PLANO_CODEX.md e rode os comandos de verificação. Não modifique regras de negócio sem registrar o motivo. Ao concluir, relate arquivos alterados, testes, pendências e próximo passo.

Prompt por etapa

> Implemente SOMENTE a Etapa N de docs/07_PLANO_CODEX.md, atendendo RF/RN/CA correspondentes. Preserve a arquitetura e não acrescente features fora de P0. Rode testes específicos, lint/typecheck e build quando aplicável. Mantenha logs de evidência de IA e descreva o que foi realmente verificado. Não invente dados nem declare sucesso sem execução.

Prompt de correção

> Corrija a falha [DESCREVER] sem reescrever módulos não relacionados. Primeiro reproduza com teste automatizado, identifique a causa, altere o menor conjunto de arquivos, execute testes regressivos e registre o antes/depois em docs/evidencias-ia/.

Prompt de auditoria

> Compare a implementação com os itens P0 de docs/02_REQUISITOS_REGRAS.md e os critérios de docs/06_TESTES_QUALIDADE.md. Gere tabela requisito → arquivo → evidência/teste → status apenas com evidência verificável, sinalize lacunas e priorize as críticas. Não implemente mudanças até identificar a causa e o escopo.

7.6 Controle de tokens e contexto

- **Uma etapa por conversa/tarefa** para reduzir difusão de mudanças; criar commits verificáveis.
- Colocar decisões permanentes em docs e AGENTS.md; não repetir toda a história do brainstorming a cada prompt.
- Exigir que o Codex leia **somente** documentos relevantes a cada etapa após o onboarding inicial.
- Não solicitar reescrita total para corrigir uma função.
- Usar testes de relógio falso, seed controlado, logs de transações e respostas de API para depuração rápida.

- Evitar novos pacotes e serviços sem necessidade clara; manter npm, monorepo enxuto e Nest modular.
- Documentar suposições como ADR curto em docs/decisoes/ antes de divergir de regras P0.

7.7 Fluxo Git sugerido

- main: estável; branches pequenas por etapa se desejado.
- Commits descritivos: chore: scaffold monorepo, feat(auth): session login, feat(focus): durable sessions, feat(review): scheduled recall, etc.
- Antes de cada merge: testes pertinentes, typecheck, migração aplicável e relatório curto de regressões.
- Não versionar .env, credenciais, dados pessoais, vídeos de clientes ou snapshots com informações reais.

7.8 Evidência do desenvolvimento com GenAI

Criar docs/evidencias-ia/README.md, além de registros selecionados com:

Campo	Conteúdo
Etapa	qual módulo foi assistido pela IA
Objetivo	requisito ou defeito real
Prompt	solicitação efetivamente realizada
Intervenção	sugestões e alterações do Codex, com decisão humana
Arquivos	módulos concretamente alterados
Verificação	comandos e saídas dos testes
Limitações	o que ainda não foi implementado ou verificado
Evidência visual	captura legível sem dados sensíveis

Não inventar conversas de IA, screenshots, commits ou testes. Comentários em código podem mencionar assistência de IA quando pertinente, mas **não** inserir comentários falsos do tipo “gerado por IA” em todo arquivo.

7.9 Critério para solicitar a etapa seguinte

Só avançar quando a etapa anterior tem build executável, migrações aplicáveis, testes essenciais e resumo claro do que foi implementado. Exceção: bloqueio externo documentado; nesse caso, registrar a pendência e não anunciar conclusão da etapa.

08 — REQUISITOS DO WORKSHOP E PLANO DE ENTREGA

8.1 Fonte e distinção

Este capítulo resume as exigências do **Final Project Submission Guide — Gen AI Workshop / E2 Proficiency / Learning Path 6618**, disponibilizado pelo usuário. São **requisitos da avaliação**, distintos dos requisitos de produto P0/P1/P2. As demais escolhas de arquitetura e experiência em docs/ são propostas deste projeto, não exigências do guia.

O material de origem identifica-se como **Confidential TCS**; evitar publicar ou redistribuir o guia em repositório público ou vídeo. Este resumo é usado apenas para conferir a entrega.

8.2 Requisitos extraídos do guia

Código	O que a avaliação exige	Como este projeto deve demonstrar
WS-01	Resolver situação real do cotidiano	dificuldade de iniciar/retomar estudos e manter revisões
WS-02	Ser validável por qualquer pessoa	dados de exemplo e instruções reproduzíveis
WS-03	Mostrar claramente onde GenAI foi usada	log verdadeiro de prompts, decisões, commits, capturas e testes
WS-04	Funcionar durante a apresentação	executar cenário completo localmente ou em hospedagem preparada
WS-05	Entregar o código-fonte	código React/Nest/Prisma e instruções de execução
WS-06	Entregar documentação da solução, objetivo e tecnologias	este pacote + README do projeto implementado
WS-07	Entregar vídeo narrado mostrando a aplicação em funcionamento	gravação real MP4/MKV/MOV ou link de vídeo privado permitido
WS-08	Enviar **ZIP** pelo formulário da Talent Development dentro do prazo comunicado	preparar pacote conforme nome/estrutura determinados
WS-09	Pasta de submissão deve ter nome ID_NAME	substituir por identificação solicitada pelo formulário, **sem inventar ID**
WS-10	Pasta deve conter **Code, Documentation, Video/URL**	verificar estrutura final abaixo
WS-11	Formulário pede endereço onde o projeto está hospedado	incluir URL pública acessível, se exigida no momento da submissão
WS-12	Compartilhamento deve permitir acesso/baixar conteúdo	testar link em janela anônima antes de submeter
WS-13	Não expor dados de TCS/cliente, dados sensíveis ou chaves	capturas e seeds completamente fictícios; .env.example sem segredos
WS-14	Respeitar políticas internas e evitar temas impróprios	conteúdo educativo neutro e dados de teste sem material restrito

Observações: o guia enfatiza evidência visível de uso de IA durante o **desenvolvimento**; não exige LLM integrada ao funcionamento do aplicativo. Não há necessidade técnica de pagar por API de geração de texto para satisfazer WS-03. O prazo concreto de entrega depende do comunicado da equipe responsável; não foi fornecido nesta especificação.

8.3 Estrutura de entrega final sugerida

```
ID_NAME/
  Code/
    estudo-em-dia/        # código-fonte, configs e migrações
  Documentation/
    README.md             # objetivo, stack, como executar
    docs/                 # esta documentação técnica
    evidencias-ia/        # registros reais, capturas e prompts
    roteiro-demonstracao.md
  Video/
```

```
  demonstracao-narrada.mp4    # ou arquivo com URL e orientações
links.txt                     # URL funcional e URLs públicas, se solicitado
```

A pasta acima representa **estrutura recomendada para submissão**; este pacote contém apenas a documentação de desenvolvimento. O código executável e o vídeo deverão ser produzidos **quando a implementação existir**, não devem ser simulados ou apresentados como prontos.

8.4 Roteiro da demonstração narrada

1. Mostrar brevemente o problema cotidiano que o produto resolve e a tecnologia empregada.
2. Entrar com conta fictícia e exibir dashboard.
3. Criar disciplina livre, por exemplo “História da Arte”, para mostrar universalidade.
4. Iniciar uma sessão de foco curta com **relógio de demonstração controlado por testes/seed**, sem falsificar atividade real em produção.
5. Pausar, retomar, concluir e explicar tempo registrado e XP (com evidência em API/log se necessário).
6. Registrar conteúdo estudado com notas curtas e ligar revisão periódica.
7. Abrir a fila com uma revisão de exemplo **pré-populada pela seed**, autoavaliar e exibir o próximo vencimento.
8. Salvar ponto de retorno; sair e entrar novamente para mostrar persistência.
9. Exibir a Academia Viva e explicar os estágios desbloqueados; usar usuário demo em estágio adequado para a gravação.
10. Mostrar brevemente evidências reais de prompts do Codex, alterações e testes executados.
11. Encerrar com limitações honestas: o app não verifica foco cognitivo nem corrige conteúdo de qualquer disciplina.

8.5 Checklist de entrega

- ☐ Código executável dentro de `Code/`, sem arquivos de segredos.
- ☐ Documentação consistente com a versão realmente implementada.
- ☐ Migrações e seed fictícia reproduzíveis, com instruções claras.
- ☐ Vídeo narrado do aplicativo **operando**, não apenas slides/mockups.
- ☐ Uso de GenAI documentado por evidências verificáveis (não inventadas).
- ☐ Link de hospedagem informado e testado no formulário, se aplicável.
- ☐ Pasta ZIP com nomenclatura `ID_NAME` real e subpastas exigidas.
- ☐ Nenhum dado corporativo, pessoal ou chave de acesso nos vídeos/arquivos.
- ☐ Arquivo ZIP testado para extração e integridade.
- ☐ Submissão realizada dentro do prazo comunicado pela Talent Development.

8.6 Não conformidades a evitar

- Submeter apenas uma descrição sem aplicativo funcional.
- Incluir somente print ou layout estático e afirmar que funciona.
- Declarar uso de IA sem registros verificáveis.
- Inventar URL ou credenciais de ambiente público.
- Apresentar dados reais de cliente, empresa ou usuários no vídeo.
- Pressupor que este **ZIP de documentação** já equivale ao **ZIP de entrega final** com código e vídeo.

09 — RASTREABILIDADE, DECISÕES E PONTOS EM ABERTO

9.1 Fluxos de alto valor e seus requisitos

Fluxo	Requisitos funcionais	Regras principais	Testes
Cadastro/entrada	RF-001–005	RN-001–005	TS-01–06
Disciplina livre	RF-010	RN-010 e RN-003	TS-02, CA-003
Conteúdo livre + revisão	RF-011–015	RN-011–013, RN-040–043	TR-01–02, TR-12–15
Foco + XP	RF-020–028	RN-020–030, RN-060–065	TF-01–15
Ponto de retorno	RF-030–034	RN-014–016	CA-015
Revisão + XP	RF-040–047, RF-051	RN-040–050	TR-03–11
Academia decorativa	RF-050–054	RN-060–063	CA-016
Estatísticas	RF-055	RN-064–065	CA-017; testes de agregação
Submissão workshop	WS-01–14	seção 08	checklist da seção 08

9.2 Decisões definitivas nesta especificação

1. Universalidade: nenhum conteúdo ou catálogo obrigatório; o próprio usuário define as disciplinas.
2. A atividade principal é uma sessão de foco registrada com recompensa após conclusão.
3. O registro de conteúdos é **manual e opcional**, inclusive depois da sessão.
4. Revisões partem desses conteúdos; não há banco de questões interno.
5. A revisão utiliza tentativa de recordar seguida de autoavaliação, não correção automática.
6. Pontos de retorno preservam **onde parei** e **próximo passo**, com armazenamento no backend.
7. A Academia Viva é somente decorativa e cresce por XP; não há gestão de cidade em P0.
8. Backend NestJS e PostgreSQL com Prisma formam uma aplicação real, não SPA apenas local.
9. Sem dependência de LLM/tokens para uso do produto; IA generativa apenas auxilia desenvolvimento e documentação da avaliação.

9.3 Parâmetros propostos, ainda não validados por usuários

Parâmetro	Padrão de implementação	Como alterar com segurança
Mínimo foco para XP	5 minutos	constante versionada + testes
Duração de foco	5–120 min	validações DTO/UI e testes de limite
XP foco	1 XP/min inteiro, até 60/sessão, 180/dia	motor de RewardEvent; migrar lógica sem alterar histórico
XP revisão	5/tentativa, teto 50/dia	motor de RewardEvent
Intervalos de revisão	1,3,7,14,30,60 dias	algoritmo versão 1 com histórico
Estágios de academia	0,100,350,800 XP	função de estágio + ativos visuais
Fuso inicial	America/Sao_Paulo	preferência editável pelo usuário
Duração do JWT	curta, a definir na configuração	segurança e testes de expiração/revogação

Não apresentar esses números como dados experimentais sobre retenção ou atenção. Representam **decisões de produto para permitir implementação e validação inicial**.

9.4 Questões técnicas a resolver na implementação sem alterar o produto

- Confirmar versão Node LTS e compatibilidade exata NestJS/Prisma 7 antes de instalar.

- Escolher estratégia explícita de cookie e CSRF em ambiente de desenvolvimento de duas portas versus produção no mesmo site.
- Definir mecanismo confiável de exclusividade de sessão aberta (índice parcial SQL ou transação com lock adequado).
- Implementar idempotência para finalização e tentativas de revisão, com testes de corrida.
- Definir auditoria mínima de segmentos de foco. A versão P0 exige tempo correto, mesmo que a tabela opcional `FocusSegment` seja adiada.
- Definir formato definitivo de respostas de paginação e do tratamento de erros de validação, consistente em todos os endpoints.
- Selecionar biblioteca de timezone e operação com `DATE` que não dependa de soma de milissegundos.

Quando a decisão afetar mais de um módulo, registrar um **ADR** no repositório com contexto, escolha, alternativas e consequências.

9.5 Limitações explícitas

- Não há verificação objetiva de concentração apenas por cronômetro.
- Não há comprovação de que o usuário aprendeu corretamente tudo que registra.
- O agendamento de revisão é uma heurística inicial de produto, não um protocolo individual cientificamente otimizado.
- A academia é incentivo visual, não medida de inteligência ou saúde cognitiva.
- A hospedagem depende de ambiente configurado; não há garantia de custo zero de infraestrutura pública.

9.6 Glossário breve

- **Disciplina / Subject:** área definida pelo usuário, como “História”, “Java”, “Inglês”.
- **Conteúdo / StudyContent:** assunto ou conceito registrado manualmente; não é um exercício do sistema.
- **Sessão de foco / FocusSession:** intervalo ativo declarado e rastreado, com pausas, conclusão ou reconciliação.
- **Ponto de retorno / ReturnPoint:** contexto salvo e próxima ação para retomada.
- **Agendamento de revisão / ReviewSchedule:** estado que determina quando revisar um conteúdo.
- **Tentativa / ReviewAttempt:** ato de recuperação ativa e autoavaliação registrado com horário e resultado.
- **Evento de recompensa / RewardEvent:** registro auditável de XP originado de sessão ou revisão.
- **Academia Viva:** cena decorativa cujo estágio deriva de XP persistido.
- **P0/P1/P2:** obrigatório no MVP / desejável após MVP / evolução futura.